



ASIS-CAM: Sistema de Asistencia Docente con Reconocimiento Facial, Geocerca y Sincronización Offline-First

Documentación Técnica y Académica del Sistema y de su Evolución Reciente

Diosmel Maximiliano Núñez

Materia: Proyecto Expo

Fecha: 16/09/2026

Diosmel Maximiliano Núñez

Argentina

Índice

Nota: este índice se genera automáticamente. En Word, haga clic derecho sobre él y seleccione "Actualizar campo" para reflejar la paginación final.

Índice	2
Resumen Ejecutivo	4
Palabras clave	5
Abstract	5
Keywords	6
Introducción y Problemática	7
Objetivos.....	8
Objetivo general	8
Objetivos específicos.....	8
Marco Teórico	10
Sistemas de información educativa y control de asistencia docente	10
Reconocimiento facial como mecanismo de autenticación biométrica	10
Geolocalización y geocercas (geofencing)	11
Backend como servicio (BaaS): Supabase	11
Firestore como plataforma de publicación estática.....	12
Arquitectura de sitio estático y despliegue continuo	12
Metodología de Desarrollo	14
Arquitectura del Sistema	15
Organización del repositorio.....	15
Modelo de datos.....	16
Stack tecnológico	16
Modificaciones Recientes y Mantenimiento Evolutivo	18
Corrección de la usabilidad del mapa de geocerca en dispositivos móviles	18
Atajo de carga masiva de horario laboral	19
Migración de la fuente de datos de producción hacia el proyecto real de Supabase ..	19
Protección de credenciales previa a la exposición pública del repositorio	20
Geocerca interactiva con Leaflet y OpenStreetMap, y corrección de zona horaria en Eventos Especiales.....	21
Configuración de Firestore Hosting como plataforma de despliegue alternativa	21
Resultados y Validación	23
Manual de Usuario — Rol Docente	24
Inicio de sesión.....	24
Marcar entrada, salida o retiro anticipado	24
Marcación sin conexión a internet.....	25

Consulta de convocatorias a Eventos Especiales	25
Consulta del historial de asistencia	25
Licencias y permisos	25
Preguntas frecuentes	26
Conclusiones y Trabajo Futuro	27
Conclusiones	27
Limitaciones actuales	28
Trabajo futuro	28
Referencias	30

Diosmel Maximiliano Núñez

Resumen Ejecutivo

El presente documento constituye la documentación técnica y académica integral de ASIS-CAM, una aplicación web de control de asistencia del personal docente basada en reconocimiento facial, geolocalización obligatoria mediante geocerca y sincronización de datos offline-first. El sistema permite a un administrador registrar docentes, definir su horario laboral, convocarlos a eventos institucionales especiales y supervisar su asistencia, mientras que cada docente utiliza su propio rostro como credencial para marcar entrada, salida y retiro anticipado desde cualquier dispositivo con cámara y navegador, dentro de un radio configurable alrededor del establecimiento educativo.

La plataforma se implementa como un sitio estático de tres archivos (index.html, style.css y script.js, sin transpilación ni empaquetado, con las librerías de terceros y los modelos de reconocimiento facial autohospedados) que persiste su información en Supabase (PostgreSQL con API REST autogenerada) y se publica mediante integración continua desde GitHub hacia Netlify, conservando además una configuración alternativa de despliegue sobre Firebase Hosting. Este documento describe, con criterios de rigor académico, el marco teórico que sustenta las tecnologías empleadas, la arquitectura del sistema, la metodología de desarrollo seguida, el conjunto íntegro de modificaciones incorporadas en la etapa más reciente del proyecto —la corrección de la usabilidad del mapa interactivo de geocerca en dispositivos móviles, el atajo de carga masiva de horario laboral, la migración de la fuente de datos de producción hacia el proyecto real de Supabase y la configuración de Firebase Hosting como plataforma de publicación alternativa—, un manual de uso orientado al rol docente, y las conclusiones y líneas de trabajo futuro derivadas del análisis realizado.

Palabras clave

Control de asistencia docente, reconocimiento facial, geocerca, geolocalización, Supabase, PostgreSQL, Firebase Hosting, JavaScript, sincronización offline-first, sistemas de información educativa.

Abstract

This document constitutes the complete technical and academic documentation of ASIS-CAM, a web application for teaching staff attendance control based on facial recognition, mandatory geolocation through geofencing, and offline-first data synchronization. The system allows an administrator to register teachers, define their work schedules, summon them to special institutional events, and monitor their attendance, while each teacher uses their own face as a credential to record clock-in, clock-out, and early departure from any device with a camera and a browser, within a configurable radius around the school.

The platform is implemented as a three-file static site (index.html, style.css, and script.js, with no transpilation or bundling step, and third-party libraries and facial-recognition models self-hosted) that persists its data in Supabase (PostgreSQL with an auto-generated REST API) and is continuously deployed from GitHub to Netlify, while also keeping an alternative Firebase Hosting deployment configuration. This document describes, with academic rigor, the theoretical framework underlying the technologies used, the system architecture, the development methodology followed, the complete set of changes introduced in the most recent development stage—the fix to the mobile usability of the interactive geofence map, the bulk work-schedule shortcut, the migration of the production data source to the real Supabase project, and the Firebase Hosting alternative deployment configuration—

, a user manual aimed at the teacher role, and the conclusions and future work derived from the analysis performed.

Keywords

Teacher attendance control, facial recognition, geofencing, geolocation, Supabase, PostgreSQL, Firebase Hosting, JavaScript, offline-first synchronization, educational information systems.

Diosmel Maximiliano Núñez

Introducción y Problemática

El control manual de la asistencia del personal docente —mediante planillas de papel, relojes de fichaje mecánicos o cuadernos de firmas— es una práctica todavía extendida en numerosas instituciones educativas de la región. Este método presenta problemas conocidos: es fácil de adulterar (una persona puede firmar por otra), no ofrece visibilidad en tiempo real para la dirección del establecimiento, dificulta la generación de reportes y estadísticas, y demanda tiempo administrativo que podría destinarse a tareas pedagógicas. A esto se suma la dificultad de coordinar convocatorias a eventos institucionales especiales (actos, jornadas, capacitaciones) y de dejar constancia fehaciente de quién asistió y quién no.

ASIS-CAM surge como respuesta a esa problemática concreta. El propósito del proyecto fue construir una herramienta web accesible desde cualquier dispositivo con cámara —sin instalar aplicaciones nativas ni hardware biométrico dedicado— que identifique a cada docente por su rostro al momento de marcar su ingreso y egreso, evitando la suplantación de identidad propia del fichaje por firma o por tarjeta compartida, y que además garantice que esa marcación se realice físicamente en la institución mediante una geocerca obligatoria.

Más allá de su diseño original, el sistema atravesó una etapa reciente de mantenimiento evolutivo motivada por el uso real de la plataforma: se detectó que el mapa interactivo utilizado para configurar la geocerca no se visualizaba correctamente en pantallas de celular, se incorporó un atajo para agilizar la carga de horarios laborales repetidos, y se corrigió una divergencia crítica entre el proyecto de Supabase de prueba y el proyecto real de producción, entre otras correcciones. Documentar esa evolución con el mismo rigor que el diseño original del sistema es uno de los objetivos centrales de este trabajo, dado que —tal como se argumenta en la sección de Metodología— el mantenimiento correctivo forma parte

constitutiva del ciclo de vida de un sistema de información y no un apéndice accesorio de su documentación.

Objetivos

Objetivo general

Desarrollar y documentar técnica y académicamente el sistema web ASIS-CAM de control de asistencia docente, analizando su arquitectura de software, el marco teórico que sustenta sus componentes tecnológicos centrales (reconocimiento facial, geocerca y persistencia como servicio) y el conjunto de modificaciones incorporadas durante su etapa más reciente de mantenimiento evolutivo, a fin de dejar constancia formal del estado actual del sistema y de su viabilidad de despliegue.

Objetivos específicos

1. Describir el marco teórico correspondiente a los sistemas de información aplicados al control de asistencia docente, al reconocimiento facial como mecanismo de autenticación biométrica, a la geolocalización por geocerca y a los modelos de backend como servicio (BaaS) empleados por el sistema.
2. Analizar la arquitectura de software de ASIS-CAM, incluyendo su modelo de datos híbrido —documental y relacional— sobre Supabase/PostgreSQL, y su estrategia de persistencia local con sincronización diferida.
3. Documentar exhaustivamente el conjunto de modificaciones incorporadas en la etapa más reciente de desarrollo: la corrección de la usabilidad del mapa interactivo de geocerca en dispositivos móviles, el atajo de carga masiva de horario laboral, la migración de la fuente

de datos de producción hacia el proyecto real de Supabase, y la configuración de Firebase Hosting como plataforma de publicación alternativa.

4. Evaluar comparativamente las dos estrategias de despliegue continuo disponibles para el sistema (Netlify y Firebase Hosting) en tanto que backends de publicación de un sitio estático.
5. Elaborar un manual de usuario orientado específicamente al rol docente, que permita a este actor operar el sistema de forma autónoma sin intervención del equipo de desarrollo.
6. Identificar las limitaciones actuales del sistema, evidenciadas durante el análisis, y proponer líneas de trabajo futuro fundamentadas en esa evidencia.

Diosmel Maximiliano Núñez

Marco Teórico

Sistemas de información educativa y control de asistencia docente

Un sistema de información educativa es un conjunto organizado de componentes de software, datos y procesos destinados a apoyar la gestión académica y administrativa de una institución. Dentro de esta categoría, los módulos de control de asistencia buscan reemplazar registros en papel por evidencia digital, trazable y auditable, reduciendo el margen de error humano y el fraude por suplantación —es decir, que una persona marque la entrada de un compañero ausente—. La digitalización de este proceso administrativo repetitivo libera tiempo del personal directivo para tareas de mayor valor agregado y mejora la calidad de los datos disponibles para la toma de decisiones.

Diosmel Maximiliano Núñez

Reconocimiento facial como mecanismo de autenticación biométrica

El reconocimiento facial es una técnica biométrica que identifica a una persona a partir de rasgos geométricos de su rostro. El flujo típico consta de tres etapas: detección del rostro dentro de una imagen o cuadro de video, extracción de un conjunto de puntos de referencia (landmarks) que describen su geometría, y cálculo de un vector numérico (descriptor facial) que resume esos rasgos de forma compacta. Dos rostros se consideran pertenecientes a la misma persona cuando la distancia euclidiana entre sus descriptores es menor a un umbral definido empíricamente. En ASIS-CAM esta técnica se implementa enteramente en el navegador del usuario mediante la librería face-api.js (Vincent, 2020), que reutiliza modelos entrenados derivados de la biblioteca dlib (King, 2009), evitando así enviar imágenes del rostro a un servidor externo para su procesamiento.

Geolocalización y geocercas (geofencing)

Una geocerca (geofence) es un perímetro virtual definido sobre un área geográfica real, habitualmente un círculo o un polígono, que se emplea para disparar una acción o una restricción cuando un dispositivo entra o sale de esa área. La técnica se apoya en la determinación de la posición del dispositivo mediante la Geolocation API del navegador (Mozilla Developer Network, 2024), que combina —según disponibilidad— señal GPS, triangulación de antenas de telefonía celular y redes wifi cercanas para estimar latitud, longitud y un radio de precisión. En un caso de uso de control de asistencia, la geocerca no reemplaza a la biometría sino que la complementa: mientras el reconocimiento facial responde a la pregunta "¿quién es esta persona?", la geocerca responde a "¿esta persona está físicamente donde debería estar?", cerrando una vía de fraude que la sola biometría no cubre —fichar remotamente con el propio rostro desde un lugar distinto al establecimiento—. La implementación concreta de la geocerca en ASIS-CAM, incluyendo su representación cartográfica interactiva, se describe en detalle en la sección de Modificaciones Recientes.

Backend como servicio (BaaS): Supabase

Un backend como servicio (BaaS) es un modelo de provisión de infraestructura en el cual un proveedor externo gestiona la base de datos, la autenticación, el almacenamiento de archivos y la exposición de una API, permitiendo que el equipo de desarrollo se concentre en la lógica de la aplicación cliente. Supabase es una plataforma BaaS de código abierto construida sobre PostgreSQL que expone automáticamente una API REST (mediante PostgREST) sobre cada tabla de la base de datos, e incorpora Row Level Security (RLS) de PostgreSQL como mecanismo declarativo de control de acceso a nivel de fila (Supabase Inc., 2024a, 2024b). RLS permite definir políticas de acceso —expresadas como condiciones SQL— que se evalúan por cada fila de una tabla antes de permitir una operación de lectura o

escritura, para un rol de base de datos determinado; en el ecosistema Supabase, el rol anon representa a cualquier cliente que se conecta usando la clave pública sin haber iniciado sesión mediante Supabase Auth.

Firestore Hosting como plataforma de publicación estática

Firestore es la plataforma de desarrollo de aplicaciones de Google, de la cual Firestore Hosting es el servicio específico de publicación de contenido web estático (Google, 2024). Al igual que Netlify, Firestore Hosting distribuye los archivos del sitio a través de una red de distribución de contenido (CDN) con certificado HTTPS automático, y su configuración se declara de forma análoga mediante un archivo firestore.json que define la carpeta a publicar (public) y las reglas de reescritura de rutas (rewrites) necesarias para que la navegación interna de una aplicación de una sola página no produzca errores 404 al recargarse desde un estado distinto del inicial. El despliegue se realiza mediante la interfaz de línea de comandos oficial (Firestore CLI), autenticada contra un proyecto de Firestore concreto que se identifica en el archivo .firebaserc del repositorio. En ASIS-CAM, Firestore Hosting se incorpora como plataforma de publicación alternativa o de respaldo frente a Netlify —que continúa siendo el destino de integración continua principal desde GitHub—, tal como se detalla en la sección de Modificaciones Recientes.

Arquitectura de sitio estático y despliegue continuo

Un sitio estático es aquel cuyos archivos (HTML, CSS, JavaScript) se sirven tal cual al navegador, sin un paso de compilación (build) ni un servidor de aplicación que genere HTML dinámicamente en cada solicitud. Plataformas como Netlify o Firestore Hosting permiten conectar un repositorio de control de versiones de modo que cada cambio subido a la rama

principal dispare automáticamente una nueva publicación —integración y despliegue continuos (CI/CD)— sin intervención manual del desarrollador.

Diosmel Maximiliano Núñez

Metodología de Desarrollo

El desarrollo se llevó adelante bajo un enfoque ágil e iterativo, coherente con los valores del Manifiesto Ágil (Beck et al., 2001): se priorizó tener en todo momento una versión funcional desplegada, se incorporaron funcionalidades en incrementos pequeños (primero el fichaje básico, luego el reconocimiento facial, luego la geocerca y los eventos especiales, luego las estadísticas) y se corrigieron errores reales detectados en uso apenas se identificaban, en lugar de posponerlos a una etapa de "cierre" del proyecto.

Bajo este enfoque, el mantenimiento evolutivo y correctivo posterior al primer despliegue —descrito en detalle en la sección de Modificaciones Recientes— no se trata como un anexo secundario sino como una iteración más del mismo ciclo: cada corrección (la usabilidad del mapa en celular, la migración de la fuente de datos real, la configuración de un despliegue alternativo) se originó en una necesidad concreta detectada durante el uso de la plataforma, se implementó de forma incremental y se verificó antes de considerarse completa, replicando el mismo criterio metodológico aplicado durante el desarrollo inicial.

El control de versiones se llevó con Git, con mensajes de commit descriptivos que documentan tanto el cambio realizado como su motivación, lo cual permitió reconstruir con precisión —para este mismo documento— la cronología y el razonamiento detrás de cada modificación reciente del sistema, sin depender exclusivamente de la memoria del equipo de desarrollo.

Arquitectura del Sistema

Organización del repositorio

El repositorio se organiza como un sitio estático sin herramienta de build, con la siguiente estructura principal:

```

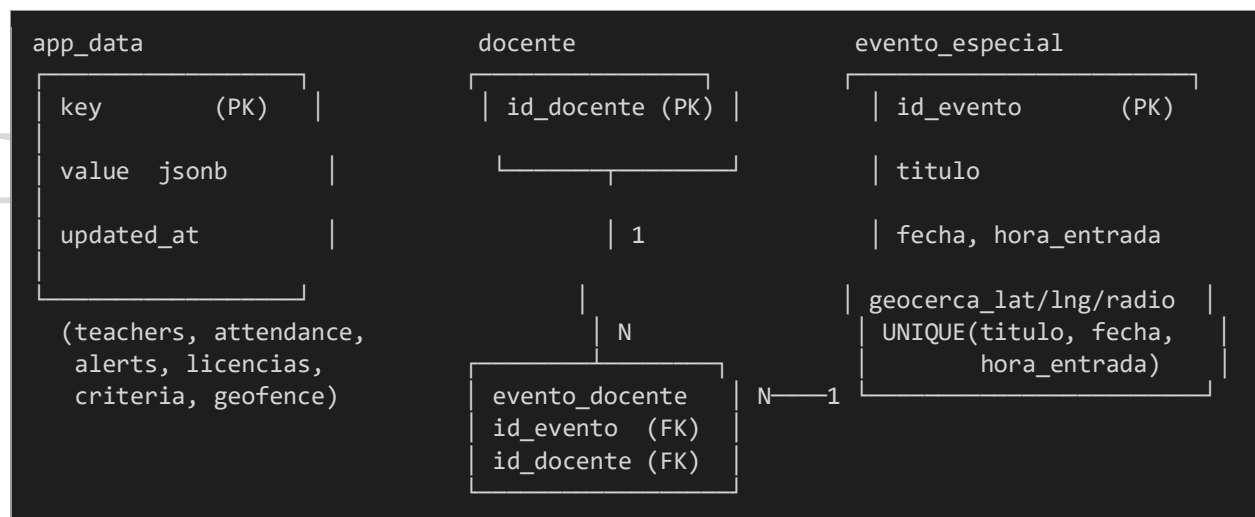
ASIS_CAM_PRO_LIMPIA/
├── index.html           # Marcado semántico + modales; sin CSS ni JS
                        # embebidos
├── style.css            # Hoja de estilos propia (tema institucional)
├── script.js            # Lógica de la aplicación, script clásico
                        # (no ES module)
├── contacto.html        # Página de contacto institucional
├── libs/                # Librerías de terceros autohospedadas
│   └── fonts/           # (Bootstrap, Bootstrap Icons + fuentes,
                        # face-api.js, Chart.js, jsPDF,
                        # supabase-js, SheetJS/xlsx)
├── models/              # 7 archivos de pesos de face-api.js
                        # autohospedados
├── sw.js                # Service worker: caché offline-first
├── supabase_schema.sql  # Creación de la tabla app_data + RLS
├── fix_rls_eventos.sql  # Políticas RLS de evento_docente/docente
├── fix_unique_evento.sql # Constraint unique_evento_dia_horario
├── add_geocerca_evento.sql # Columnas de geocerca por evento
├── migrate_kclnaabvcxdovvgblyoc_schema.sql
                        # Migración al esquema del proyecto real
├── usuarios_schema.sql  # Tabla usuarios
├── netlify.toml / _redirects # Configuración de publicación en Netlify
├── firebase.json / .firebaserc # Configuración de Firebase Hosting
├── tests/
│   ├── test_offline_sync.js # Arnés de sincronización diferida
│   └── test_service_worker.js # Arnés del service worker (sw.js)
├── documentacion/       # Informe técnico, manual y folleto previos
├── docs/                 # Esta documentación consolidada
└── package.json          # Metadatos, scripts de despliegue y "npm test"

```

La ausencia de un paso de build (no hay Webpack, Vite ni similar) es una decisión deliberada: al tratarse de una única página con navegación por paneles (mostrar/ocultar secciones del DOM), el costo de incorporar un framework de componentes no se justificaba frente a la simplicidad de mantener HTML, CSS y JavaScript planos, lo cual además simplifica el despliegue, dado que no existen artefactos de build que puedan quedar desactualizados respecto del código fuente.

Modelo de datos

El modelo de datos combina dos estrategias de persistencia dentro de la misma base PostgreSQL provista por Supabase: un almacén clave-valor genérico (la tabla `app_data`, que guarda como documentos JSON las colecciones `teachers`, `attendance`, `alerts`, `licencias`, `criterias`, `geofence`, `modoPrueba`, `kioskPrincipal` y `kioskCodes`) y un conjunto de tablas relacionales propias para el módulo de Eventos Especiales (`evento_especial`, `evento_docente` y `docente`), que sí requiere integridad referencial real. Esta convivencia no es una inconsistencia sino una decisión pragmática: el fichaje diario tiene una estructura flexible que cambió varias veces durante el desarrollo, mientras que la convocatoria a eventos necesitaba garantías de unicidad que un documento JSON no puede ofrecer por sí mismo.



Stack tecnológico

Capa	Tecnología	Rol en el sistema
Interfaz	HTML5 + CSS3 + JavaScript (ES2017, script clásico)	Marcado, estilos y lógica de negocio del cliente
Componentes de UI	Bootstrap 5.3 + Bootstrap Icons 1.10	Maquetación responsiva y modales
Mapas y geocerca	Leaflet + OpenStreetMap + Nominatim	Selección visual de la geocerca y geocodificación
Biometría	face-api.js 0.22 (sobre TensorFlow.js)	Detección y reconocimiento facial en el navegador
Gráficos	Chart.js 4.4	Estadísticas de asistencia

Capa	Tecnología	Rol en el sistema
Reportes	jsPDF 2.5 / SheetJS (xlsx)	Exportación a PDF y Excel
Persistencia	Supabase (PostgreSQL + PostgREST)	Base de datos, API REST y RLS
Offline	localStorage + Service Worker (sw.js)	Caché de datos y de assets para uso sin conexión
Despliegue principal	Netlify (CI/CD desde GitHub)	Publicación continua de producción
Despliegue alternativo	Firebase Hosting	Publicación de respaldo / entorno adicional
Pruebas	Node.js (arneses propios, módulo vm)	Verificación automatizada sin navegador
Documentación	docx (Node.js)	Generación programática de documentos APA 7

Diosmel Maximiliano Núñez

Modificaciones Recientes y Mantenimiento Evolutivo

Esta sección documenta, en orden cronológico, el conjunto íntegro de cambios incorporados al repositorio durante la etapa de mantenimiento evolutivo más reciente del proyecto, reconstruida a partir del historial de control de versiones (Git). Cada apartado indica el problema detectado, la solución implementada y su justificación técnica.

Corrección de la usabilidad del mapa de geocerca en dispositivos móviles

Se detectó que el mapa interactivo de Leaflet utilizado para configurar la geocerca — tanto en el panel de Configuración como en el formulario de Evento Especial— no se dimensionaba correctamente al abrirse desde un celular. La causa fue doble: el contenedor del mapa (`geofenceMapContainer` y `eventoGeocercaMapContainer`) fijaba su altura mediante un estilo en línea (`height:320px`) en lugar de una clase CSS, lo cual dificulta el ajuste responsivo, y la instancia de Leaflet no tenía habilitadas explícitamente las interacciones táctiles (arrastre, zoom por pellizco, doble toque).

La corrección reemplazó el estilo en línea por una clase dedicada,

```
.geocerca-map-container {  
  display: block;  
  width: 100%;  
  min-height: 380px; /* min-height, no height: no se recorta en celular */  
  border-radius: 8px;  
  overflow: hidden;  
}
```

y habilitó de forma explícita en la inicialización de Leaflet las opciones `dragging`, `touchZoom`, `tap` y `doubleClickZoom`, desactivando en cambio `scrollWheelZoom`, de modo que el desplazamiento normal de la página con el dedo no quede accidentalmente capturado por el mapa. El uso de `min-height` en lugar de `height` es la corrección determinante: permite que el contenedor crezca según el contenido disponible en pantallas angostas en vez de forzar

una altura fija que Leaflet podía calcular mal antes de que el modal terminara de mostrarse por completo.

Atajo de carga masiva de horario laboral

Se incorporó, en el formulario de alta y edición de docentes, un mecanismo para aplicar el mismo bloque horario a varios días de la semana en un solo paso: el usuario marca mediante casillas de verificación los días a los que desea aplicar el horario (lunes a domingo), completa una única hora de inicio y una única hora de finalización, y presiona "Aplicar a los días marcados". Antes de esta mejora, cargar un horario idéntico para, por ejemplo, los cinco días hábiles de la semana requería repetir manualmente la misma operación de alta de horario cinco veces. La función `aplicarHorarioRapido()` reutiliza la misma validación de horarios duplicados ya existente para el alta individual, de modo que el atajo no introduce una vía alternativa que eluda esa verificación.

Migración de la fuente de datos de producción hacia el proyecto real de Supabase

Se detectó que existían dos proyectos de Supabase distintos referenciados entre las dos líneas de desarrollo del repositorio (identificados por sus referencias de proyecto `zyxcummfswlnaupvaqor` y `kclnaabvcxdovvgblyoc`), con historiales de Git no relacionados entre sí. Mediante una verificación directa contra la API pública de ambos proyectos se constató que `kclnaabvcxdovvgblyoc` contenía el historial de datos real (8 docentes, 3 eventos especiales y 10 convocatorias, 37 fichajes registrados), mientras que `zyxcummfswlnaupvaqor` contenía únicamente datos de prueba (2 docentes, 6 fichajes).

Sobre la base de esa verificación se ejecutaron, en orden, las siguientes acciones: (1) se actualizaron las constantes `SUPABASE_URL` y `SUPABASE_ANON_KEY` en `script.js` para que la aplicación lea y escriba contra el proyecto con los datos reales; (2) se corrigieron

los comentarios de los scripts SQL del repositorio (`supabase_schema.sql`, `fix_rls_eventos.sql`, `fix_unique_evento.sql`), que todavía referenciaban el proyecto de prueba; (3) se generó y ejecutó una migración de esquema (`migrate_kclnaabvcxdovvgblyoc_schema.sql`) que llevó las tablas docente, evento_especial y evento_docente del proyecto real al esquema más nuevo que `script.js` ya esperaba —agregando, entre otras columnas, `escuela_id` y `fecha_inicio/fecha_fin` en `evento_especial`, y creando la tabla `usuarios` que faltaba— preservando la totalidad de los identificadores y vínculos existentes; y (4) se fusionaron (merge) las dos ramas de historial no relacionadas del repositorio, conservando de la rama master las funcionalidades más completas (mapa interactivo de geocerca, geocerca por evento, recuperación de contraseña de administrador, exportación a Excel, atajo de horario rápido) e incorporando de la otra rama los directorios `tests/` y `documentacion/` que no generaban conflicto.

Tras ejecutar la migración se verificó contra la API en vivo que los 8 docentes, los 3 eventos y las 10 convocatorias permanecían intactos y que las políticas de Row Level Security de escritura seguían funcionando correctamente sobre el esquema migrado, confirmando que la operación no produjo pérdida de datos.

Protección de credenciales previa a la exposición pública del repositorio

Como paso de preparación para la exhibición pública del proyecto, se removieron del árbol de trabajo los archivos de texto plano que contenían identificadores de servicio de correo (`Id_Service_gmail_js.txt`) y credenciales de recuperación de contraseña (`password_email_js.txt`), agregándolos a `.gitignore` para evitar que vuelvan a versionarse por error. En el mismo cambio se incorporó el script SQL `add_geocerca_evento.sql`, que agrega las columnas de geocerca por evento a `evento_especial`.

Geocerca interactiva con Leaflet y OpenStreetMap, y corrección de zona horaria en

Eventos Especiales

El mecanismo de geocerca, originalmente resuelto con iframes de Google Maps, se reemplazó por un mapa interactivo propio construido con Leaflet sobre teselas de OpenStreetMap, sin necesidad de una clave de API de un proveedor comercial. La implementación (funciones `initGeocercaMap()`, `setGeocercaPoint()`, `reverseGeocodeGeocerca()` y `buscarGeocercaDireccion()`, entre otras, agrupadas en el objeto `geocercaMaps`) ofrece: un buscador de direcciones que consulta el servicio de geocodificación Nominatim de OpenStreetMap (limitado a resultados de Argentina), un marcador arrastrable con un círculo que representa visualmente el radio configurado, geocodificación inversa automática al mover el marcador (para mostrar la dirección aproximada correspondiente a esas coordenadas) y un botón "Usar mi ubicación actual" que consulta la Geolocation API del navegador. La función `validarGeocerca()` impide guardar la configuración de geocerca —tanto la general como la de un Evento Especial— si no se seleccionó un punto válido en el mapa.

En el mismo cambio se corrigió un defecto de zona horaria en el módulo de Eventos Especiales: las columnas `fecha_inicio` y `fecha_fin` de `evento_especial` son de tipo `timestamp` sin zona horaria en PostgreSQL, pero el código construía esos valores con `new Date()` y los serializaba con `toISOString()`, lo cual reinterpreta la fecha y hora locales como si fueran UTC, corriendo el valor guardado varias horas respecto de lo que el usuario había ingresado. La corrección arma `fecha_inicio` y `fecha_fin` como cadenas de texto planas (sin pasar por `new Date()/toISOString()`) al guardar, y las relee con `split("T")` en lugar de reconstruir un objeto `Date`, eliminando la conversión de zona horaria en ambos sentidos.

Configuración de Firebase Hosting como plataforma de despliegue alternativa

Se incorporó al repositorio la configuración necesaria para publicar el sitio también en Firebase Hosting, como alternativa o respaldo del despliegue principal en Netlify: el archivo `firebase.json` define `public: "."` (se sirve la raíz del repositorio, igual que en Netlify) y una regla de reescritura de cualquier ruta hacia `/index.html`, necesaria por el mismo motivo que la regla equivalente de `netlify.toml` —evitar errores 404 al recargar la aplicación de una sola página desde un estado interno de navegación—. El archivo `.firebaserc` asocia el repositorio con el proyecto de Firebase concreto que recibirá la publicación (identificado en el estado actual del repositorio como `asistencia-docente-test-27800`, en reemplazo del marcador de posición `REPLACE_WITH_YOUR_FIREBASE_PROJECT_ID` con el que se había dejado preparado el archivo). Esta configuración deja al proyecto en condiciones de ejecutar `firebase deploy --only hosting` (script ya declarado en `package.json`) para publicar una copia del sitio en la infraestructura de Google, de forma independiente del flujo de integración continua de Netlify.

Diosmel Maximiliano Núñez

Resultados y Validación

El sistema cuenta con dos arneses de pruebas automatizadas, ejecutables con npm test sin depender de un navegador ni de hardware de cámara: tests/test_offline_sync.js, que ejecuta el script.js real dentro de un contexto aislado de Node.js con implementaciones simuladas de localStorage y de un cliente de Supabase conmutable entre "conectado" y "desconectado"; y tests/test_service_worker.js, que simula el entorno de un Service Worker y ejecuta sw.js real dentro de él. La última ejecución combinada registrada arrojó 36 verificaciones sobre 36 aprobadas.

Componente verificado	Mecanismo de prueba	Resultado
Sincronización diferida offline-first	Arnés Node.js sobre script.js real	16/16 aprobadas
Precacheo y estrategias de caché del service worker	Arnés Node.js sobre sw.js real	20/20 aprobadas
Restricción de unicidad de eventos (fix_unique_evento.sql)	Inserción manual duplicada en Supabase	Rechazada por PostgreSQL, como se esperaba
Migración de esquema al proyecto real	Consulta directa a la API tras migrar	8 docentes, 3 eventos y 10 convocatorias intactos
Geocerca interactiva en mapas móviles	Verificación manual en dispositivo celular	Contenedor y gestos táctiles operativos

Un hallazgo relevante detectado durante el análisis, documentado con más detalle en el informe técnico previo del proyecto (documentacion/01_Informe_Tecnico_ASIS_CAM_APA7.docx), es que las políticas de Row Level Security de las tablas app_data, evento_especial, evento_docente y docente están definidas de forma abierta para el rol anon, dado que el sistema no utiliza Supabase Auth: la separación entre administrador y docente es, en el estado actual, exclusivamente una capa de presentación en el cliente, no una restricción a nivel de base de datos. Este punto se retoma en la sección de Conclusiones y Trabajo Futuro.

Manual de Usuario — Rol Docente

Esta sección explica, paso a paso, cómo un docente utiliza ASIS-CAM. Requiere un dispositivo con cámara (computadora, tablet o celular), un navegador actualizado (Chrome, Edge o Safari) y el DNI y la contraseña entregados por la administración.

Inicio de sesión

1. Abrir el navegador y acceder a la dirección de la plataforma.
2. Ingresar el número de DNI como usuario y la contraseña asignada.
3. Presionar el botón "Ingresar". Se recomienda cambiar la contraseña provisoria desde "Mi perfil" → "Cambiar contraseña" en el primer inicio de sesión.

Marcar entrada, salida o retiro anticipado

1. En el panel personal, presionar el botón "Identificarme".
2. Mirar de frente a la cámara, sin gorra ni lentes oscuros, hasta que el sistema confirme la identidad mediante la comparación de tres muestras faciales sucesivas.
3. Una vez identificado, presionar el botón correspondiente: "Entrada", "Salida" o "Retiro anticipado". Solo se habilita el botón que corresponde al momento del día y al horario laboral cargado.

El sistema exige una identificación facial nueva antes de cada marcación —no reutiliza una identificación previa de la misma sesión— para impedir que un docente registre la asistencia de otro. Además de la identificación biométrica, la marcación se valida contra la geocerca configurada: si el dispositivo está fuera del radio permitido, la marcación queda bloqueada salvo en el escenario de modo avión sin señal GPS descrito más abajo.

Marcación sin conexión a internet

Si el dispositivo pierde la conexión a internet, ASIS-CAM sigue permitiendo marcar la asistencia: el reconocimiento facial funciona completamente offline (los modelos están autohospedados y precacheados por el service worker) y el registro se guarda localmente en el dispositivo, mostrando un aviso de color amarillo que indica que se subirá automáticamente al recuperar la señal. Si, además de no haber conexión, el GPS tampoco responde a tiempo, el sistema no bloquea la marcación: la guarda igualmente, marcada como pendiente de validación de geocerca, y la valida contra la geocerca vigente en cuanto el dispositivo recupera la conexión.

Consulta de convocatorias a Eventos Especiales

1. En la sección "Mis eventos", revisar los eventos institucionales a los que se fue convocado.
2. El día del evento, marcar presente identificándose por rostro de la misma forma que en el fichaje diario.

Consulta del historial de asistencia

1. Ingresar a "Mi historial".
2. Consultar, mes por mes, las entradas, salidas, tardanzas y faltas registradas, junto con el porcentaje de asistencia del período.

Licencias y permisos

Las licencias (ausencias autorizadas) las carga la administración. Un docente que necesite una licencia debe comunicarse con la dirección del establecimiento; una vez cargada, se reflejará en su calendario personal y esos días dejarán de computarse como falta.

Preguntas frecuentes

El sistema no reconoce el rostro. Verificar que haya buena iluminación sobre el rostro, quitarse lentes oscuros o gorra, y volver a presionar "Identificarme". Si el problema persiste, solicitar a la administración que vuelva a capturar las fotos de referencia.

Se marcó "Retiro anticipado" por error. Comunicarse con la administración: puede revisar el registro en el buzón de alertas y, si corresponde, autorizarlo o dejar la aclaración correspondiente.

No hay cámara disponible ese día. Informar a la administración, que puede registrar la asistencia manualmente desde el panel de administrador como excepción, dejando constancia del motivo.

Diosmel Maximiliano Núñez

Conclusiones y Trabajo Futuro

Conclusiones

ASIS-CAM cumple su objetivo central: reemplazar el fichaje manual de asistencia docente por un mecanismo biométrico que dificulta la suplantación de identidad, complementado con una geocerca que exige la presencia física del docente en el establecimiento, corriendo enteramente en el navegador y sin requerir hardware dedicado. El análisis realizado para este documento confirma que la arquitectura de sitio estático combinada con un backend como servicio sostiene funcionalidad no trivial —reconocimiento facial, geolocalización, detección automática de faltas, convocatoria a eventos, sincronización offline-first— sin necesidad de un framework de frontend ni de un servidor de aplicación propio.

La revisión del mantenimiento evolutivo más reciente muestra un patrón consistente: cada corrección incorporada (la usabilidad del mapa en celular, la migración a la fuente de datos real de producción, la geocerca interactiva con Leaflet y OpenStreetMap, la corrección de zona horaria en Eventos Especiales y la habilitación de Firebase Hosting como despliegue alternativo) se originó en una necesidad concreta detectada durante el uso real de la plataforma y no en una revisión especulativa de código, lo cual es consistente con el enfoque ágil e iterativo declarado como metodología de desarrollo.

El caso de la migración de datos, en particular, deja como aprendizaje que la existencia de dos historiales de Git no relacionados apuntando a dos proyectos de backend distintos es un riesgo operativo concreto —no meramente teórico— capaz de hacer que una aplicación en producción escriba silenciosamente sobre datos de prueba en lugar de datos reales; su resolución mediante verificación directa contra la API antes de decidir qué

proyecto conservar es una práctica generalizable a cualquier situación de divergencia de configuración entre ramas.

Limitaciones actuales

- La precisión de la geocerca depende del permiso de geolocalización otorgado por el navegador y de la calidad de la señal GPS/red del dispositivo; en interiores o con mala señal puede generar falsos rechazos.
- Las políticas de Row Level Security de `app_data`, `evento_especial`, `evento_docente` y `docente` están abiertas para el rol `anon`, dado que el sistema no utiliza Supabase Auth: la separación `admin/docente` existe únicamente en el cliente, no en la base de datos.
- La sincronización diferida `offline-first` sincroniza cada colección como un documento completo; si dos dispositivos guardan cambios distintos sobre la misma colección estando ambos sin conexión, el que sincroniza en segundo lugar sobrescribe por completo lo que había subido el primero.
- La configuración de Firebase Hosting incorporada es, al momento de este documento, una vía de despliegue alternativa configurada pero no la vía de integración continua principal, que sigue siendo Netlify desde GitHub.

Trabajo futuro

- Migrar la autenticación a Supabase Auth y restringir las políticas de RLS al rol `authenticated`, de modo que la separación `admin/docente` exista también a nivel de base de datos.
- Registrar y revisar periódicamente la tasa de rechazos de fichaje por geocerca, para calibrar el radio configurado con datos de uso real.

- Rediseñar la sincronización diferida para registrar cada fichaje o alerta como una fila individual con marca de tiempo, en lugar de un documento completo por colección, de modo que sincronizar sea agregar filas nuevas en vez de reemplazar un documento.
- Evaluar la promoción de Firebase Hosting a un esquema de despliegue redundante activo (por ejemplo, como entorno de contingencia ante una caída de Netlify), documentando el procedimiento de conmutación entre ambos.

Diosmel Maximiliano Núñez

Referencias

- American Psychological Association. (2020). Publication manual of the American Psychological Association (7th ed.). <https://doi.org/10.1037/0000165-000>
- Agafonkin, V. (2024). Leaflet: An open-source JavaScript library for interactive maps [Software]. <https://leafletjs.com/>
- Beck, K., Beedle, M., van Bennekum, A., Cockburn, A., Cunningham, W., Fowler, M., Grenning, J., Highsmith, J., Hunt, A., Jeffries, R., Kern, J., Marick, B., Martin, R. C., Mellor, S., Schwaber, K., Sutherland, J., & Thomas, D. (2001). Manifesto for Agile Software Development. <https://agilemanifesto.org/>
- Chart.js Contributors. (2024). Chart.js documentation (v4.4). <https://www.chartjs.org/docs/latest/>
- Google. (2024). Firebase Hosting documentation. <https://firebase.google.com/docs/hosting>
- King, D. E. (2009). Dlib-ml: A machine learning toolkit. *Journal of Machine Learning Research*, 10, 1755–1758.
- Mozilla Developer Network. (2024). Geolocation API. https://developer.mozilla.org/es/docs/Web/API/Geolocation_API
- Mozilla Developer Network. (2024). Service Worker API. https://developer.mozilla.org/es/docs/Web/API/Service_Worker_API
- Netlify, Inc. (2024). Netlify Docs: Continuous deployment. <https://docs.netlify.com/site-deploys/create-deploys/>
- OpenStreetMap Foundation. (2024). Nominatim: Geocoding service using OpenStreetMap data. <https://nominatim.org/>
- OpenStreetMap Foundation. (2024). OpenStreetMap. <https://www.openstreetmap.org/>

PostgreSQL Global Development Group. (2024). PostgreSQL 16 documentation: Row security policies. <https://www.postgresql.org/docs/current/ddl-rowsecurity.html>

PostgREST. (2024). PostgREST documentation. <https://postgrest.org/en/stable/>

Supabase Inc. (2024a). Supabase documentation. <https://supabase.com/docs>

Supabase Inc. (2024b). Row Level Security.

<https://supabase.com/docs/guides/database/postgres/row-level-security>

Twitter, Inc. / Bootstrap Team. (2023). Bootstrap 5.3 documentation.

<https://getbootstrap.com/docs/5.3/>

Vincent, J. [justadudewhohacks]. (2020). face-api.js: JavaScript face recognition API for the browser and Node.js, implemented on top of TensorFlow.js core [Software]. GitHub.

<https://github.com/justadudewhohacks/face-api.js>

World Wide Web Consortium. (2022). Service Workers. [https://www.w3.org/TR/service-](https://www.w3.org/TR/service-workers/)

[workers/](https://www.w3.org/TR/service-workers/)

Diosmel Maximiliano Núñez